

# Stochastic Gaussian Splat Rendering Denoising using Lightweight Autoencoders

M. Bregar<sup>1</sup>

<sup>1</sup>University of Ljubljana, Faculty of Computer and Information Science, Slovenia

## Abstract

*Stochastic visibility sampling provides an efficient alternative to depth-sorted alpha blending for Gaussian splat rendering, but produces noisy RGB and depth frames with visible temporal flickering. We present a lightweight recurrent U-Net-style autoencoder for denoising such stochastic renders. The model propagates hidden states across frames to exploit temporal information and reconstruct stable RGB outputs. We evaluate non-recurrent, recurrent, and confidence-augmented variants on seen and unseen Gaussian splat scenes. The results show that temporal recurrence improves reconstruction quality and temporal consistency, while the tested confidence-map setup does not improve performance.*

## CCS Concepts

•Computing methodologies → Neural networks; Image processing;

## 1. Introduction

Rendering semi-transparent Gaussian splats typically requires depth sorting followed by alpha blending, which can be costly for dynamic views. Stochastic visibility sampling provides an alternative by sampling fragments with a probability according to their projected opacity. Each fragment is treated as a binary visibility sample, and averaging repeated independent samples yields a Monte Carlo estimate of the alpha-composited image.

This formulation avoids explicit per-view depth sorting and replaces alpha blending with simple stochastic visibility tests. However, individual frames contain substantial sampling noise. The resulting RGB and depth images vary with the randomly selected fragments, producing pixel-level noise and temporal flickering that are unsuitable for direct interactive display.

We address this by formulating frame denoising as an end-to-end learned reconstruction problem. Our method uses a lightweight recurrent autoencoder that takes a noisy stochastic render and auxiliary per-pixel signals as input and predicts a temporally stable denoised image suitable for real-time visualization.

## 2. Related Work

A substantial body of work addresses image denoising using deep neural networks. In this work, we focus on lightweight approaches capable of denoising sequences of noisy frames in real time.

*Interactive Reconstruction of Monte Carlo Image Sequences using a Recurrent Denoising Autoencoder.* Chaitanya et al. [CKS\*17] propose a recurrent denoising autoencoder for low-sample Monte

Carlo image sequences. Their method uses a U-Net-style autoencoder with recurrent blocks that propagate hidden states between frames, improving temporal stability. The network also supports auxiliary per-pixel buffers such as depth and normals. Our model follows this general design, using a recurrent autoencoder structure with renderer-derived auxiliary inputs.

*Real-time Monte Carlo Denoising with the Neural Bilateral Grid.* Meng et al. [MZV\*20] introduce a real-time neural denoiser for 1 spp Monte Carlo renderings based on a differentiable bilateral grid. Their network learns a guide image for splatting and slicing noisy samples in bilateral space, combining end-to-end learning with efficient edge-aware filtering. This provides a lightweight alternative to direct autoencoder-based reconstruction.

*Real-time Monte Carlo Denoising with Weight Sharing Kernel Prediction Network.* Fan et al. [FWHB21] propose a real-time kernel-prediction denoiser for very low-sample Monte Carlo path-traced images. Instead of predicting full per-pixel kernels directly, their method predicts a compact kernel-map encoding and reconstructs filtering kernels with an efficient decoder. This represents another efficient approach to real-time learned denoising.

## 3. Method

### 3.1. Problem Formulation

Unlike Monte Carlo path-tracing denoising, where noise often appears as sparse high-energy outliers, stochastic splat rendering produces dense binary sampling noise from plausible splat colors. As a result, noisy pixels may remain close to the local color distribution, making them difficult to identify from RGB values alone.

The available auxiliary information is also limited. Gaussian splat scenes encode position, scale, rotation, opacity, and color, but do not directly provide explicit surfaces, surface normals, UV parameterizations, material textures, or object-level geometry. The denoising method must therefore operate with restricted and potentially unreliable guidance signals.

We assume static Gaussian splat scenes with camera motion only, so temporal variation comes from viewpoint changes and stochastic sampling. The denoiser is restricted to renderer-derived inputs: noisy RGB and noisy depth.

### 3.2. Denoising Model Architecture

We use a recurrent U-Net-style autoencoder with five encoder and decoder stages, as shown in Figure 1. The network receives the renderer-derived input tensor and predicts a denoised RGB image.

Each encoder stage applies convolutional feature extraction followed by a recurrent convolutional update using the hidden state from the previous frame. Hidden states are maintained independently at each encoder scale, allowing temporal information to propagate at multiple spatial resolutions. Starting from a base width of 24 channels, the number of feature channels increases by a factor of 4/3 at each encoder stage:

$$C_i = \left\lceil 24 \left( \frac{4}{3} \right)^i \right\rceil, \quad i \in \{0, \dots, 4\}. \quad (1)$$

The decoder upsamples the bottleneck representation and concatenates the corresponding recurrent encoder features through skip connections. The final decoder feature map is mapped to RGB using a  $1 \times 1$  convolution. We predict a residual correction added to the input RGB and constrain the output to  $[0, 1]$  using a sigmoid:

$$I_{\text{denoised}} = \sigma(f(x) + I_{\text{noisy}}). \quad (2)$$

### 3.3. Training

The recurrent model is trained on short sequences generated by the stochastic renderer. Each training sample contains  $N = 7$  consecutive frames, and a random  $128 \times 128$  crop is extracted during dataset loading. Hidden states are initialized to zero at the beginning of each sequence and propagated through time using truncated backpropagation.

The training objective combines spatial reconstruction, gradient consistency, and temporal consistency. The spatial term is an  $L_1$  loss between the prediction  $\hat{I}_t$  and the target image  $I_t$ :

$$L_{\text{spatial}}(t) = |\hat{I}_t - I_t|. \quad (3)$$

The gradient term applies a fixed Laplacian-of-Gaussian filter  $G$  to both images:

$$L_{\text{gradient}}(t) = |G(\hat{I}_t) - G(I_t)|. \quad (4)$$

Temporal consistency is enforced by matching frame-to-frame differences between predictions and targets:

$$L_{\text{temporal}} = \frac{1}{N-1} \sum_{t=1}^{N-1} |(\hat{I}_t - \hat{I}_{t-1}) - (I_t - I_{t-1})|. \quad (5)$$

Spatial and gradient losses are accumulated over the sequence using a fixed Gaussian ramp that assigns larger weights to later frames. The complete sequence loss is

$$L_{\text{sequence}} = \lambda_s L_s + \lambda_g L_g + \lambda_t L_{\text{temporal}}, \quad (6)$$

where  $L_s$  and  $L_g$  denote the weighted 7-frame sequence Gaussian weighted averages of the spatial and gradient losses, and  $\lambda_s$ ,  $\lambda_g$ , and  $\lambda_t$  are the corresponding loss weights.

### 3.4. Inference Pipeline

The inference pipeline combines WebGPU rendering passes with an ONNX inference backend, allowing the denoising system to run in a web browser. As shown in Figure 2, each frame is rendered into noisy RGB and depth textures, packed into an ONNX input tensor, and processed together with the recurrent hidden states. The network outputs a denoised RGB tensor and updated hidden states and the RGB output is then converted back into a GPU texture and composited to the screen.

### 4. Training Setup

Training was performed on three enclosed Gaussian splat scenes: a backyard garden [Ost26], an office space [Kyu26], and an indoor gymnasium [Eth26], shown in Figure 3. These scenes provide broad spatial coverage, making random patch extraction more reliable.

Training data was generated by rendering each frame twice: once with the stochastic splat renderer to obtain noisy RGB and depth inputs, and once with depth-sorted alpha blending to obtain the ground-truth RGB target. All rendering, training and inference was performed on 720p images.

Camera paths were generated automatically by rotating around each scene and inserting short pauses to expose the model to both camera motion and stationary stochastic sampling. We sampled 516 frames from the Garden scene, 408 from the Gymnasium scene, and 410 from the Office scene. Training examples were constructed from 30-frame sequences by extracting 7-frame subsequences. Every fifth generated sequence was used for training validation and checkpoint selection.

Table 1: Training hyperparameters.

| Parameter                                          | Value                          |
|----------------------------------------------------|--------------------------------|
| Epochs                                             | 200                            |
| Optimizer                                          | AdamW                          |
| Scheduler                                          | Cosine annealing               |
| Learning rate                                      | $10^{-3}$ to $5 \cdot 10^{-6}$ |
| Base channels                                      | 24                             |
| Loss weights ( $\lambda_s, \lambda_g, \lambda_t$ ) | (0.8, 0.1, 0.1)                |
| Sequence length                                    | 7                              |
| Patch size                                         | $128 \times 128$               |

All training and inference tests were performed on a machine with an Intel i5-13600K CPU and an AMD RX 7900 XTX GPU.

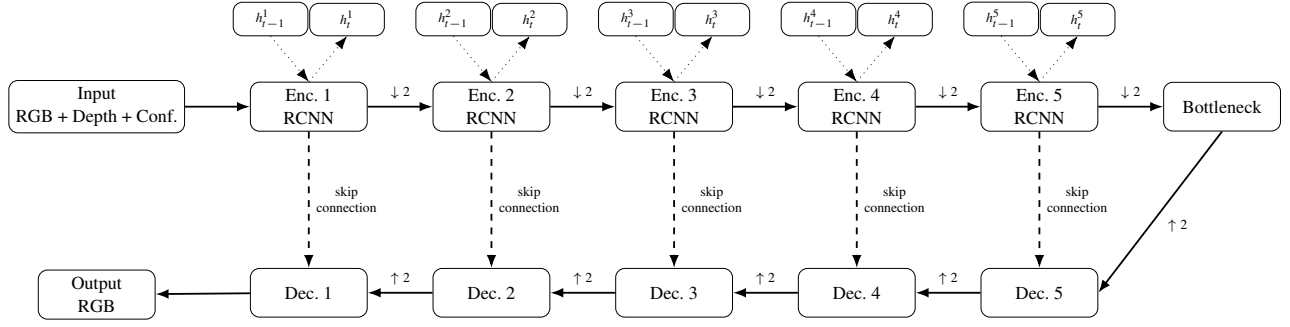


Figure 1: Recurrent U-Net style denoising autoencoder extended with an auxiliary confidence map input. Each encoder stage receives a stage-specific hidden state from the previous frame and outputs the corresponding updated hidden state. Encoders downsample the image by a factor of 2 while preserving some of the original input via skip connections. The decoder upsamples the bottleneck features and concatenates the encoder skip features to preserve high frequency detail.

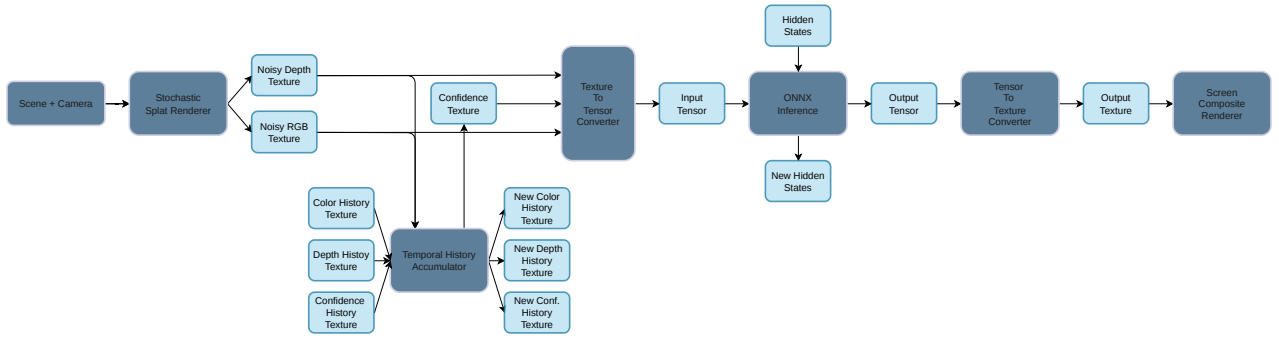
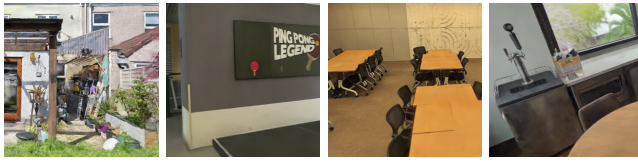


Figure 2: Browser inference pipeline. Stochastic RGB and depth textures are packed into an ONNX tensor and processed by the recurrent denoiser together with hidden states. The confidence texture path is used only by the confidence-augmented model variant.



(a) Garden (b) Gym (c) Office (d) Kitchen

Figure 3: Depth-sorted alpha-blended renders of all scenes.

## 5. Evaluation Setup

We evaluate three U-Net-style denoising models: a non-recurrent five-stage U-Net, a recurrent model using noisy RGB and depth inputs, and a recurrent model with an additional confidence-map input.

The recurrent models were trained on two datasets: Garden only and the full three-scene dataset. The non-recurrent U-Net was trained only on Garden.

Each model was evaluated on separate 200-frame sequences. The Garden evaluation sequence measures performance on a seen training scene, while the Kitchen sequence [Sma26] was excluded from training and evaluates cross-scene generalization.

We compute RMSE and SSIM as the primary evaluation metrics. RMSE measures pixel-wise reconstruction accuracy while penalizing larger errors more strongly, whereas SSIM evaluates structural similarity between the denoised output and the target rendering. Together, these metrics capture both numerical reconstruction error and perceptual image quality.

### 5.1. Confidence Map Experiment Setup

We extend the inference pipeline with a temporal history confidence accumulator. The confidence-augmented denoiser receives noisy RGB, noisy depth, and a normalized confidence map. The confidence value represents the effective number of reliable temporal observations supporting each pixel:

$$\hat{C}_t(x) = \frac{C_t(x)}{C_{\max}}. \quad (7)$$

For each valid current-frame pixel  $x$ , depth  $D_t(x)$  is used to reconstruct its world-space position, which is reprojected into the previous frame to obtain the history location  $x_{t-1}$ . We then sample the previous color history  $H_{t-1}^C$ , depth history  $H_{t-1}^D$ , and confidence  $C_{t-1}$  at this location.

History is reused only when it is consistent with the current



Figure 4: Visualization of confidence map. Dark pixels represent lower confidence due to stochastic sampling noise and camera motion.

frame. We compute a usability weight from validity checks, depth agreement, reprojection distance, and color similarity:

$$U_t(x) = V_t(x) M_D(x) M_R(x) M_I(x). \quad (8)$$

Here,  $V_t(x)$  contains binary validity checks, while  $M_D(x)$ ,  $M_R(x)$ , and  $M_I(x)$  are depth, reprojection-distance, and color-stability multipliers in  $[0, 1]$ .

The carried confidence is computed by applying the usability weight and temporal decay factor  $\lambda$  to the previous confidence:

$$C_{t-1}^{\text{carry}}(x) = \lambda U_t(x) C_{t-1}(x_{t-1}). \quad (9)$$

Each valid current pixel contributes one new observation:

$$O_t(x) = \begin{cases} 1, & \text{if } D_t(x) \text{ is not background,} \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

The updated confidence is the sum of usable carried confidence and current observation support, clamped to a fixed maximum:

$$C_t(x) = \min \left( C_{t-1}^{\text{carry}}(x) + O_t(x), C_{\max} \right). \quad (11)$$

The carried confidence also determines how strongly the previous color and depth histories are reused. The blend factor is

$$\alpha = \frac{C_{t-1}^{\text{carry}}(x)}{\max(C_t(x), \epsilon)}. \quad (12)$$

The accumulated histories are updated as:

$$H_t^I(x) = (1 - \alpha) I_t(x) + \alpha H_{t-1}^I(x_{t-1}), \quad (13)$$

$$H_t^D(x) = (1 - \alpha) D_t(x) + \alpha H_{t-1}^D(x_{t-1}). \quad (14)$$

The normalized confidence map  $\hat{C}_t(x)$  is then provided to the denoiser as an additional input channel. A visualization of the confidence texture is shown in Figure 4.

## 6. Results

The quantitative evaluation results are shown in Table 2.

The recurrent models outperform the non-recurrent U-Net on

Table 2: Evaluation metrics for different models, training datasets, and evaluation datasets.

| Model             | Train set             | Eval set | RMSE     | SSIM     |
|-------------------|-----------------------|----------|----------|----------|
| U-Net             | Garden                | Kitchen  | 0.036869 | 0.870538 |
| Recurrent         | Garden                | Kitchen  | 0.029394 | 0.894751 |
| Recurrent         | Garden + Gym + Office | Kitchen  | 0.025123 | 0.901342 |
| Recurrent + Conf. | Garden                | Kitchen  | 0.032959 | 0.878226 |
| Recurrent + Conf. | Garden + Gym + Office | Kitchen  | 0.067347 | 0.824029 |
| U-Net             | Garden                | Garden   | 0.066580 | 0.623725 |
| Recurrent         | Garden                | Garden   | 0.054047 | 0.703935 |
| Recurrent         | Garden + Gym + Office | Garden   | 0.054202 | 0.696430 |
| Recurrent + Conf. | Garden                | Garden   | 0.055749 | 0.694458 |
| Recurrent + Conf. | Garden + Gym + Office | Garden   | 0.082186 | 0.652743 |

both evaluation sequences, showing the benefit of temporal information. The best result on Kitchen is achieved by the recurrent model trained on Garden + Gym + Office, while the best result on Garden is achieved by the recurrent model trained only on Garden.

The confidence-map input does not improve performance. In all cases, confidence-augmented models produce higher RMSE and lower SSIM than the corresponding recurrent models without confidence input.

### 6.1. Qualitative Results

Figure 6 shows a qualitative comparison on the Kitchen evaluation sequence. The non-recurrent U-Net produces smoother outputs, but introduces substantial blur and noticeable color inaccuracies. The recurrent model without confidence produces sharper reconstructions and better preserves scene structure, although all evaluated models still reduce some high-frequency detail relative to the reference.

The confidence-map variant does not show a clear visual improvement over the recurrent RGB-depth model, matching the quantitative results in Table 2.

Figure 5 shows a qualitative comparison for one frame from the Garden sequence. The visual trends are broadly consistent with those observed in Figure 6. However, the difference between the default recurrent model and the confidence-augmented model is less visually apparent in this example, despite the substantial difference in their quantitative scores.

### 6.2. Runtime Performance and Model Size

The final recurrent model contains 849,513 trainable parameters. Since denoising inference dominates the runtime of the rendering pipeline, we report inference-only performance for several model and backend configurations in Table 3. Rendering and other pipeline stages are excluded from the measurements.

## 7. Discussion

The results indicate that temporal recurrence is beneficial for stochastic splat denoising. Since the renderer produces different sampling noise in each frame, recurrent hidden states allow the





Figure 5: Qualitative comparison on the Garden evaluation sequence. From left to right, the columns show the noisy stochastic Gaussian splat rendering, the denoised outputs produced by the evaluated models, and the target depth-sorted alpha-blended rendering.

Table 3: Inference-only runtime for the evaluated denoising configurations. FPS denotes average processed frames per second, and frame time is reported in milliseconds.

| Metric          | Rec. | Rec. + Conf. | Rec. + Conf. FP16 | PyTorch | Compiled PyTorch |
|-----------------|------|--------------|-------------------|---------|------------------|
| Average FPS     | 21.1 | 20.7         | 26.1              | 45.9    | 138.3            |
| Frame time (ms) | 47.3 | 48.2         | 38.2              | 21.7    | 7.2              |

model to accumulate information over time, improving reconstruction quality and temporal consistency compared to frame-independent inference.

The confidence-map input did not improve performance. Although it was designed to represent per-pixel reliability, this signal may not directly correspond to reconstruction usefulness. The recurrent model may already infer similar information from RGB and depth, making the explicit confidence channel redundant or difficult to exploit.

The multi-scene-trained model improves performance on the unseen Kitchen sequence, achieving lower RMSE and higher SSIM than the Garden-only recurrent model. This suggests that adding scenes does improve generalization. However, on the Garden evaluation sequence, the Garden-only model remains slightly better, indicating that scene-specific training can still be advantageous when desiring maximal quality.

All trained models still exhibit noticeable blurring, especially in high-frequency regions. This suggests that the current loss function does not sufficiently penalize missing fine detail. Future work should explore stronger gradient supervision, perceptual losses, or other objectives better suited for preserving sharp structure.

Runtime measurements show that the inference backend is the main practical bottleneck. While browser-based ONNX inference provides broad compatibility, it substantially reduces performance compared to native PyTorch execution. Backend-specific optimization, half-precision inference or WebGPU-specific model design could improve real-time performance.

## 8. Conclusion

We presented a lightweight recurrent U-Net-style denoiser for stochastic Gaussian splat rendering. The model reconstructs stable

RGB frames from noisy stochastic renders using auxiliary depth information and recurrent hidden states.

The results show that temporal recurrence improves reconstruction quality and temporal consistency compared to frame-independent denoising. Adding an explicit confidence map did not improve performance, and the tested multi-scene training setup did not improve generalization.

Future work should focus on reducing residual blur, improving cross-scene generalization with larger datasets, and optimizing the inference backend for real-time browser deployment.

## References

- [CKS\*17] CHAITANYA C. R. A., KAPLANYAN A. S., SCHIED C., SALVI M., LEFOHN A., NOWROUZEZAHRAI D., AILA T.: Interactive reconstruction of monte carlo image sequences using a recurrent denoising autoencoder. *ACM Transactions on Graphics* 36, 4 (2017), 1–12. doi:10.1145/3072959.3073601. 1
- [Eth26] ETHAN: Table tennis room. <https://superspl.at/scene/0d9e613b>, 2026. Gaussian splat scene. Licensed under CC BY 4.0: <https://creativecommons.org/licenses/by/4.0/>. 2
- [FWHB21] FAN H., WANG R., HUO Y., BAO H.: Real-time monte carlo denoising with weight sharing kernel prediction network. *Computer Graphics Forum* 40, 4 (2021), 15–27. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.14338>, arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14338>, doi:10.1111/cgf.14338. 1
- [Kyu26] KYUMIN L.: Gunja514. <https://superspl.at/scene/d152f2df>, 2026. Gaussian splat scene. Licensed under CC BY 4.0: <https://creativecommons.org/licenses/by/4.0/>. 2
- [MZV\*20] MENG X., ZHENG Q., VARSHNEY A., SINGH G., ZWICKER M.: Real-time Monte Carlo Denoising with the Neural Bilateral Grid. In *Eurographics Symposium on Rendering - DL-only Track* (2020), Dachsbaier C., Pharr M., (Eds.), The Eurographics Association. doi:10.2312/sr.20201133. 1
- [Ost26] OSTROMECKI M.: Garden. <https://superspl.at/scene/52ef6279>, 2026. Gaussian splat scene. Licensed under CC BY 4.0: <https://creativecommons.org/licenses/by/4.0/>. 2
- [Sma26] SMALL A.: Coworking kitchen. <https://superspl.at/scene/4294afa9>, 2026. Gaussian splat scene. Licensed under CC BY 4.0: <https://creativecommons.org/licenses/by/4.0/>. 3

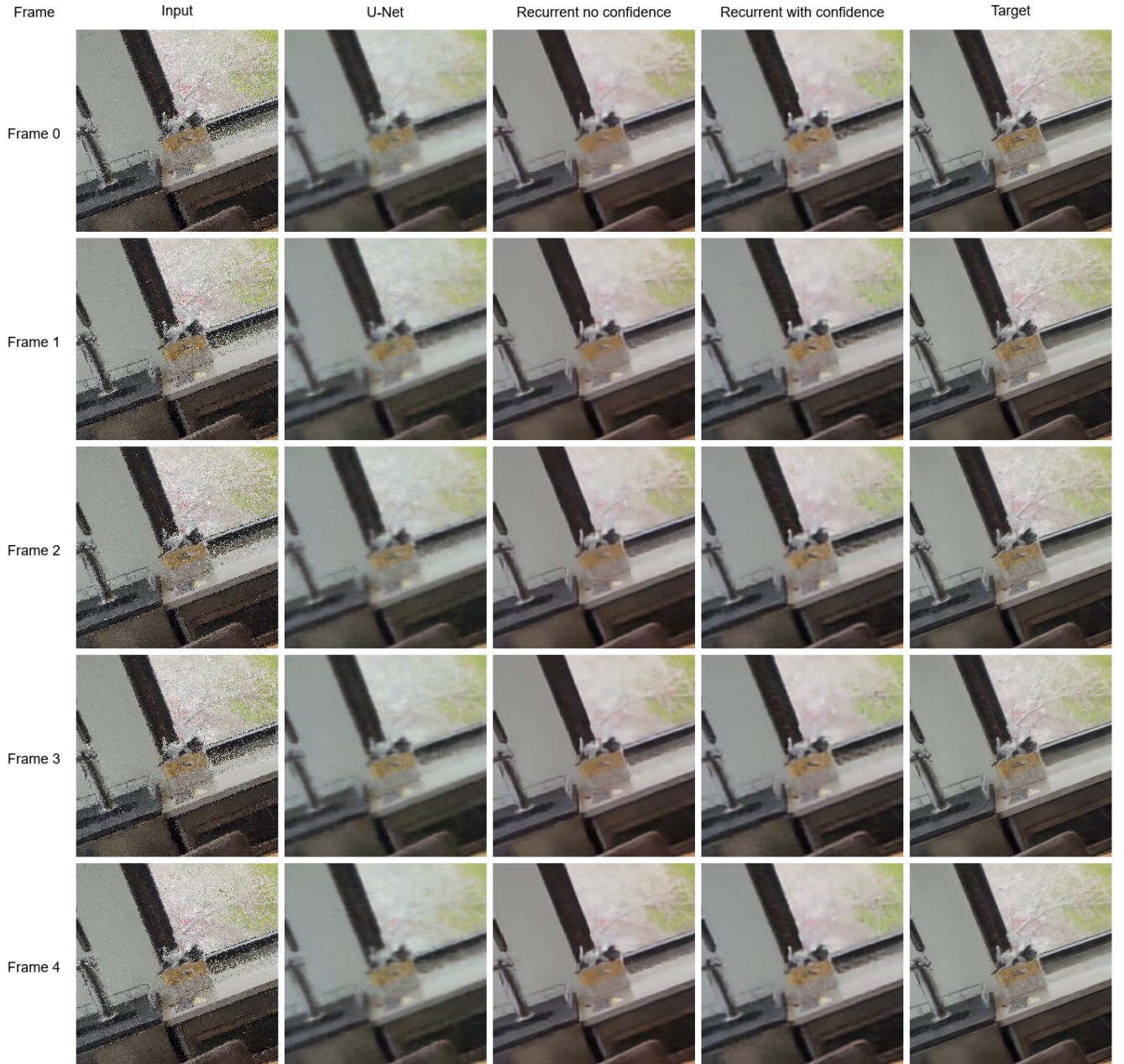


Figure 6: Qualitative comparison on the Kitchen evaluation sequence. The non recurrent U-Net model was trained on purely the Garden scene, while the recurrent and confidence extended recurrent models were trained on the full 3-scene dataset. Rows show consecutive frames, while columns compare the noisy input, model outputs, and reference image for the same frame.